

7. 应用层之HTTP

1. 应用层概念

应用层是计算机网络体系结构的最顶层，是设计和建立计算机网络的最终目的，也是计算机网络中发展最快的部分。包括：

早期基于文本的应用（电子邮件、远程登录、文件传输、新闻组），20世纪90年代将因特网带入千家万户的万维网WWW，当今流行的即时通信、P2P文件共享及各种音视频应用。



1.1. 应用层协议的定义

应用层协议 (application layer protocol) 定义了在不同端系统上应用程序是如何相互传输报文的。一般来说，应用层协议会规定如下内容：

- 交换的报文类型：交换的是请求报文还是响应报文。
- 报文字段的解释：对报文中各个字段的详细描述。
- 报文字段的语义：报文各个字段的含义是什么。
- 报文交换时间、方式：程序何时、以什么方式发送报文以及响应。

直白一点来说，应用层只是产生和使用数据的逻辑层，在这一层次我们并不会关心它们是如何发送数据的以及数据发到哪里。应用层有两种层次结构。

2. 应用层体系结构

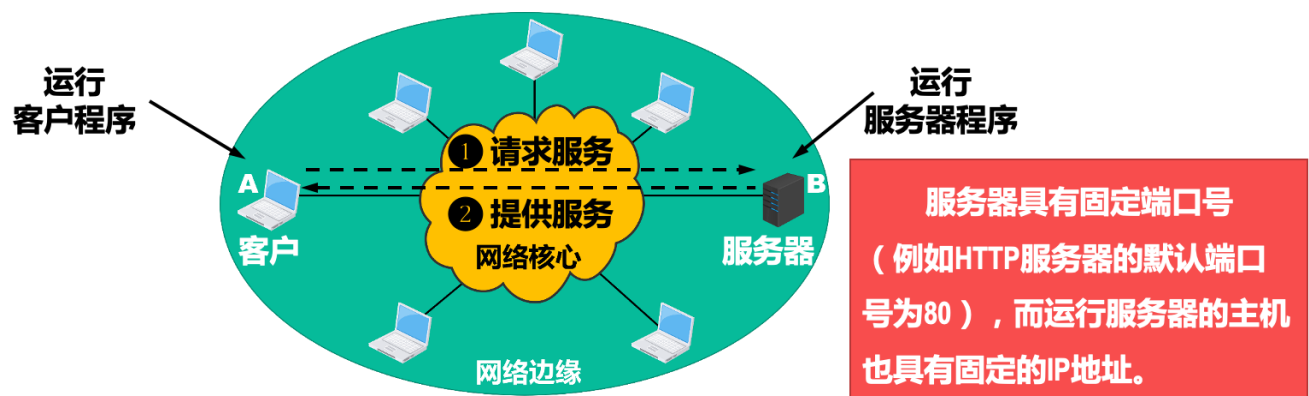
应用层体系结构 (Application Architecture) 定义了应用层端系统之间数据交换的方式，开发一种新的网络应用，首先要考虑的问题就是网络应用程序在各种端系统上的组织方式和它们之间的关系一般来说，主流的体系结构有两种：

- 客户-服务器体系结构 (client-server architecture)
- 对等体系结构 (P2P architecture)

2.1. 客户/服务器方式

在客户-服务器体系结构中，分为请求方和服务方。有一个总是打开的主机称为服务端 (Server)，它向客户端 (client) 提供服务。客户端会发送请求给服务端，服务端会根据客户端的请求做出响应。

- 体系特点
 - 客户和服务器是指通信中所涉及的两个应用进程。
 - 客户/服务器方式所描述的是进程之间服务和被服务的关系。
 - 客户是服务请求方，服务器是服务提供方。
 - 服务器总是处于运行状态，并等待客户的服务请求。



我们最常见的服务端就是Web服务器，Web服务器提供于来自浏览器的请求。我们日常访问百度、谷歌，其实就是在访问它们的Web服务器。

一般常见的 Web 服务器主要有Apache、IIS、Jboss、Tomcat、WebSphere、WebLogic 等。

当 Web 服务器通过浏览器接收到用户请求后，它会经过一系列的处理把信息或者页面等通过浏览器呈现给用户。这种模式就是客户 - 服务器模式。

这里有两点需要注意：

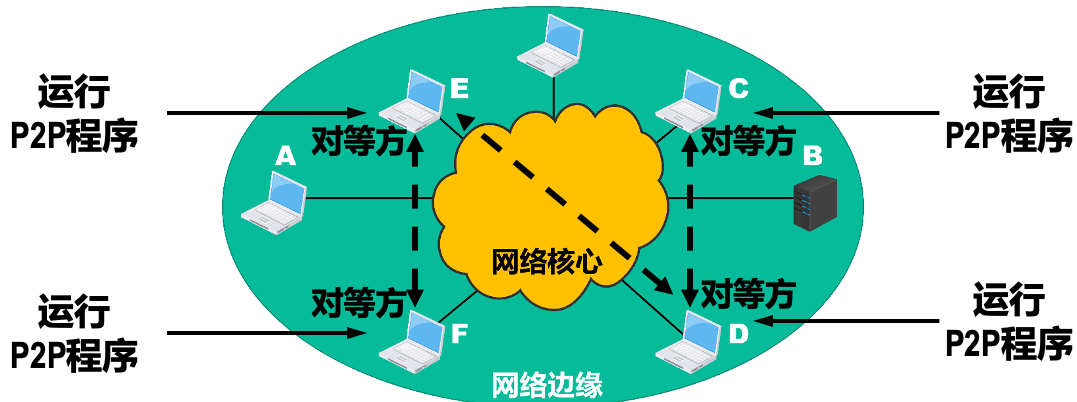
- 在客户 - 服务器模式下，通常客户彼此之间是并不互相通信的。
- 服务器通常具有固定的、周知的 IP 地址可以提供访问。

客户 - 服务器模式通常会出现随着客户数量的急剧增加导致单台服务器无法满足大量请求的情况。为此，通常需要具备大量的数据中心 (data center)，用来跟踪所有的用户请求。

2.2. 点对点方式

在P2P方式中，**没有固定的服务请求者和提供者**，分布在网络边缘各端系统中的应用进程是对等的，被称为**对等方**。对等方相互之间直接通信，每个对等方既是服务的请求者，又是服务的提供者。

在因特网上流行的P2P应用主要包括P2P文件共享、即时通信、P2P流媒体、分布式存储等。



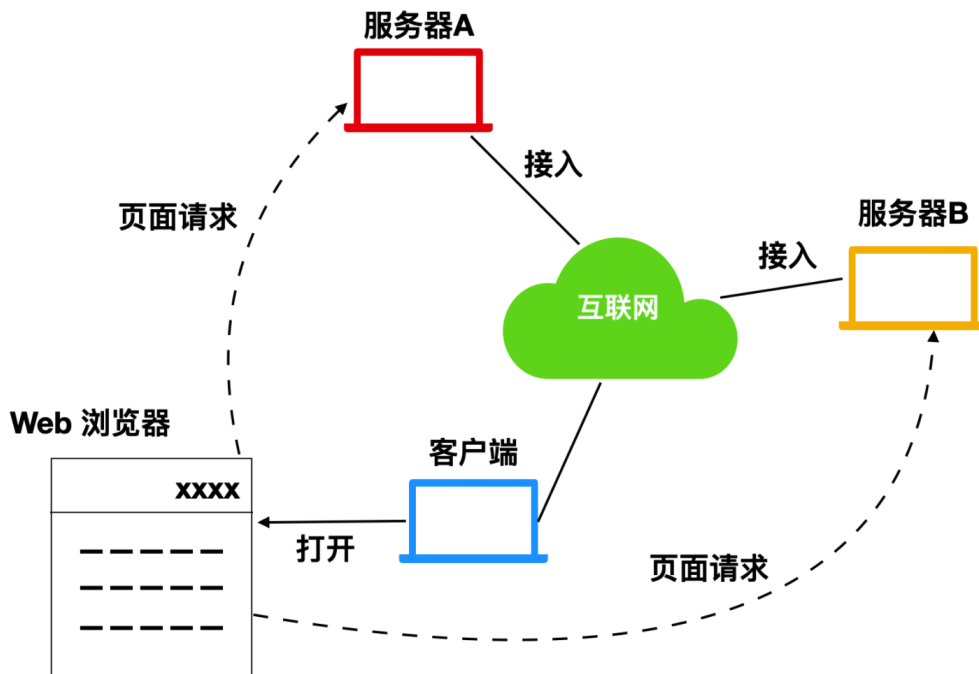
P2P 一个最大的特点就是**扩展性 (self-scalability)**，因为 P2P 网络的一个重要的目标就是让所有的客户端都能提供资源、获取资源，共享带宽，存储空间等。因此，当有更多节点加入且对系统请求增多，整个系统的容量也增大。这是具有一组固定服务器的客户 - 服务器结构不具备的，这也就是 P2P 的优势。P2P方式具有**成本上的优势**，因为它通常不需要庞大的服务器设置和服务器带宽。为了降低成本，服务提供商对于将P2P方式用于应用的兴趣越来越大。

3. 万维网

3.1. 万维网概述

万维网 (World Wide Web, WWW) **并非某种特殊的计算机网络**。它是一个大规模的、联机式的信息储藏所，是运行在因特网上的一个分布式应用。万维网利用网页之间的**超链接**将不同网站的网页链接成一张逻辑上的信息网。

万维网 (WWW, World Wide Web) 是将互联网中的信息以超文本的形式展现的系统，用来显示 WWW结果的客户端被称为 Web 浏览器。通过浏览器，我们无需关注想要访问的内容在哪个服务器上，我们只需要知道我们想访问的内容就可以了。



WWW 定义了三个比较重要的概念，这些概念主要有：

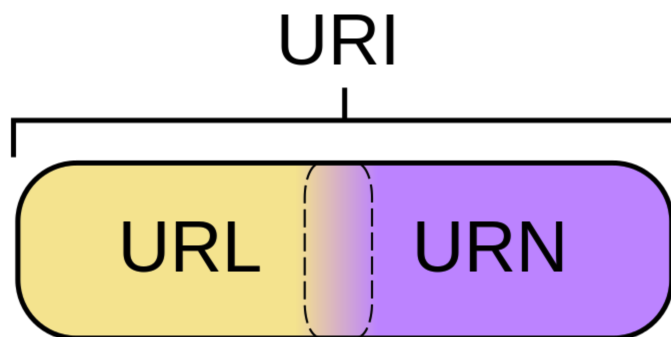
- URI定义了访问信息的手段和位置
- 文档规范
- HTTP 定义了 WWW 的访问规范

3.2. 统一资源定位符

URI (Uniform Resource Identifier) 中文名称是统一资源标识符，使用它能够唯一地标记互联网上资源。

URL (Uniform Resource Locator) 中文名称是统一资源定位符，也就是我们俗称的网址，它实际上是URI 的一个子集。

URI不仅包括URL，还包括URN（统一资源名称），它们之间的关系如下：



URI 已经不局限于标识互联网资源，它可以作为所有资源的识别码。URL的一般形式由以下四个部分组成：

< 协议 > : // < 主机 > : < 端口 > / < 路径 >

3.3. 万维网文档

3.3.1. HTML

超文本标记语言 (HyperText Markup Language, HTML), 使用多种“标签”来描述网页的结构和内容。HTML 称为超文本标记语言, 是一种标识性的语言。它包括一系列标签。通过这些标签可以将网络上的文档格式统一, 使分散的Internet资源连接为一个逻辑整体。HTML文本是由HTML命令组成的描述性文本, HTML命令可以说明文字、图形、动画、声音、表格、链接等。



3.3.2. CSS

层叠样式表 (Cascading Style Sheets, CSS), 从审美的角度来描述网页的样式。

```
.pink {  
  color: ■ deppink;  
  font-size: 36px;  
}
```

3.3.3. JavaScript

一种脚本语言, 控制网页的行为。

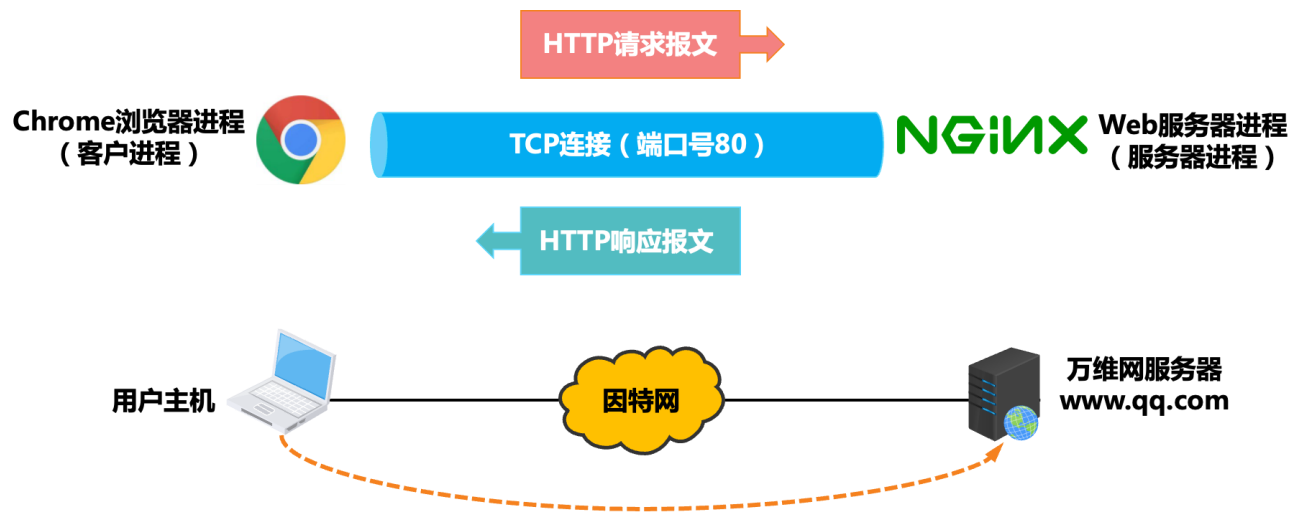
```
function myFunction() {  
  document.getElementById("myId").innerHTML="谢谢你的赞";  
}
```

Demo.js

3.3.4. HTTP

Web 的应用层协议就是 HTTP (HyperText Transfer Protocol, HTTP)， 超文本传输协议，它是 Web 的核心协议。下面是 HTTP 协议中的几个核心概念。

HTTP定义了浏览器（即万维网客户进程）怎样向万维网服务器请求万维网文档，以及万维网服务器怎样把万维网文档传送给浏览器。



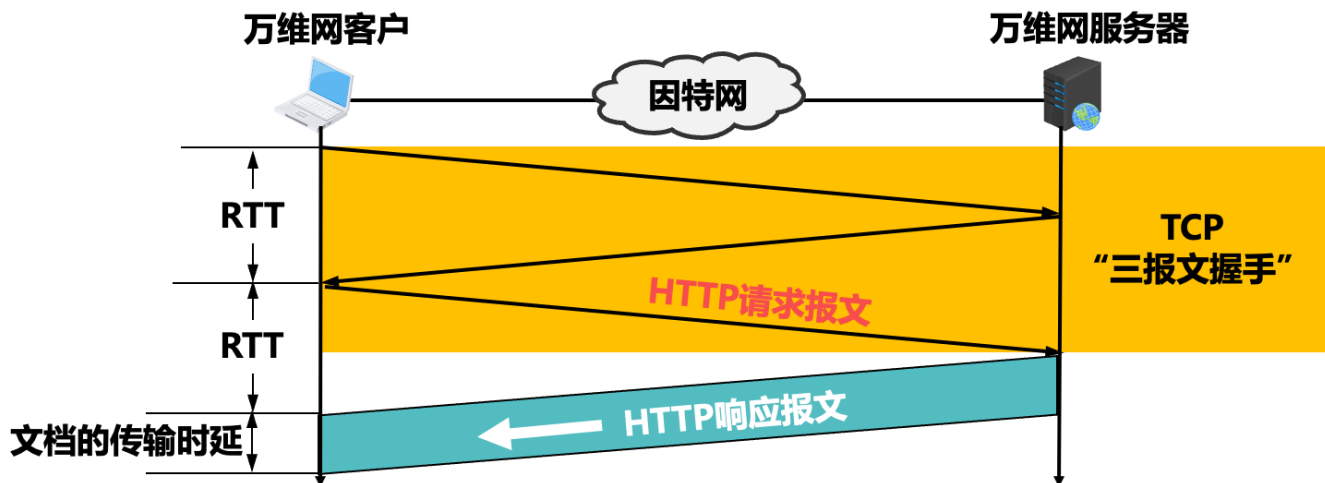
4. HTTP

HTTP 是一个在计算机世界里专门在两点之间传输文字、图片、音频、视频等超文本数据的约定和规范。HTTP 是一种应用层协议，它使用TCP作为运输层协议，因为文档、数据这些信息在我们看来是一种重要的信息，不可丢失。

4.1. HTTP连接行为

4.1.1. HTTP/1.0

HTTP/1.0采用非持续连接方式。在该方式下，每次浏览器要请求一个文件都要与服务器建立TCP连接，当收到响应后就立即关闭连接。



每请求一个文档就要有两倍的RTT的开销。若一个网页上有很多引用对象（例如图片等），那么请求每一个对象都需要花费2RTT的时间。

为了减小时延，浏览器通常会建立多个并行的TCP连接同时请求多个对象。但是，这会大量占用万维网服务器的资源，特别是万维网服务器往往要同时服务于大量客户的请求，这会使其负担很重。

4.1.2. HTTP/1.1

HTTP/1.1采用持续连接方式。在该方式下，万维网服务器在发送响应后仍然保持这条连接，使同一个客户（浏览器）和该服务器可以继续在这条连接上传送后续的HTTP请求报文和响应报文。这并不局限于传送同一个页面上引用的对象，而是只要这些文档都在同一个服务器上就行。

为了进一步提高效率，HTTP/1.1的持续连接还可以使用流水线方式工作，即浏览器在收到HTTP的响应报文之前就能够连续发送多个请求报文。这样的一个个的请求报文到达服务器后，服务器就发回一个接一个的响应报文。这样就节省了很多个RTT时间，使TCP连接中的空闲时间减少，提高了下载文档的效率。

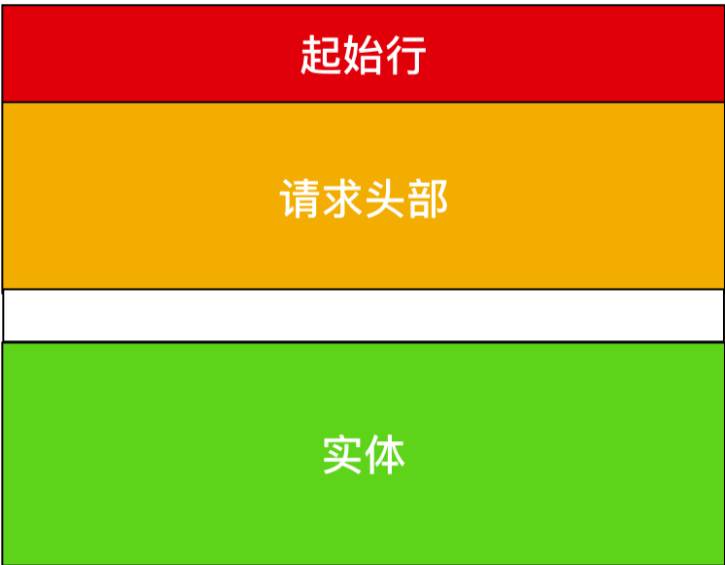
4.2. HTTP报文格式

我们上面描述了一下 HTTP 的请求响应过程，相信你对 HTTP 有了更深的认识，下面我们就来一起认识一下 HTTP 的报文格式是怎样的。

HTTP 协议主要由三大部分组成：

- 起始行（start line）：描述请求或响应的基本信息。
- 头部字段（header）：使用 key-value 形式更详细地说明报文。
- 消息正文（entity）：实际传输的数据，它不一定是纯文本，可以是图片、视频等二进制数据。

其中起始行和头部字段并称为请求头或者响应头，统称为 Header；消息正文也叫做实体，称为 body。HTTP 协议规定每次发送的报文必须要有 Header，但是可以没有 body，也就是说头信息是必须的，实体信息可以没有。而且在 header 和 body 之间必须要有一个空行（CRLF）。如果用一幅图来表示一下 HTTP 请求的话，我觉得应该是下面这样：



4.2.1. HTTP请求

这幅图需要注意一下，如果使用 GET 方法，是没有实体体的，如果你使用的是 POST 方法，才会有实体体。当用户提交表单时，HTTP 客户端通常使用 POST 方法；与此相反，HTML 表单的获取通常使用 GET 方法。HEAD 方法类似于 GET 方法，只不过 HEAD 方法不会返回对象。



方 法	描 述
GET	请求URL标志的文档
HEAD	请求URL标志的文档的首部
POST	向服务器发送数据
PUT	在指明的URL下存储一个文档
DELETE	删除URL标志的文档
CONNECT	用于代理服务器
OPTIONS	请求一些选项信息
TRACE	用来进行环回测试
PATCH	对PUT方法的补充，用来对已知资源进行局部更新

4.2.2. HTTP响应



状态码 (五大类33种)	描 述
1XX	表示通知信息，如请求收到了或正在进行处理；
2XX	表示成功，如接受或知道了；
3XX	表示重定向，即要完成请求还必须采取进一步的行动；
4XX	表示客户的差错，如请求中有错误的语法或不能完成；
5XX	表示服务器的差错，如服务器失效无法完成请求。

4.3. HTTP协议特征

从上面整个过程中我们可以总结出 HTTP 进行分组传输是具有以下特征：

- 支持客户 – 服务器模式
- 简单快速：客户向服务器请求服务时，只需传送请求方法和路径。请求方法常用的有 GET、HEAD、POST。每种方法规定了客户与服务器联系的类型不同。由于 HTTP 协议简单，使得 HTTP 服务器的程序规模小，因而通信速度很快。

- 灵活：HTTP 允许传输任意类型的数据对象。正在传输的类型由 Content-Type 加以标记。
- 无连接：无连接的含义是限制每次连接只处理一个请求。服务器处理完客户的请求，并收到客户的应答后，即断开连接。采用这种方式可以节省传输时间。
- 无状态：HTTP 协议是无状态协议。无状态是指协议对于事务处理没有记忆能力。缺少状态意味着如果后续处理需要前面的信息，则它必须重传，这样可能导致每次连接传送的数据量增大。另一方面，在服务器不需要先前信息时它的应答就较快。

4.4. cookie和session

HTTP 协议是一种无状态协议，即每次服务端接收到客户端的请求时，都是一个全新的请求，服务器并不知道客户端的历史请求记录；Session 和 Cookie 的主要目的就是为了弥补 HTTP 的无状态特性。

4.4.1. cookies是什么

HTTP 协议中的 Cookie 包括 Web Cookie 和浏览器 Cookie，它是服务器发送到 Web 浏览器的一小块数据。服务器发送到浏览器的 Cookie，浏览器会进行存储，并与下一个请求一起发送到服务器。通常，它用于判断两个请求是否来自于同一个浏览器，例如用户保持登录状态。

HTTP Cookie 机制是 HTTP 协议无状态的一种补充和改良。

Cookie 主要用于下面三个目的：

- 会话管理

登陆、购物车、游戏得分或者服务器应该记住的其他内容。

- 个性化

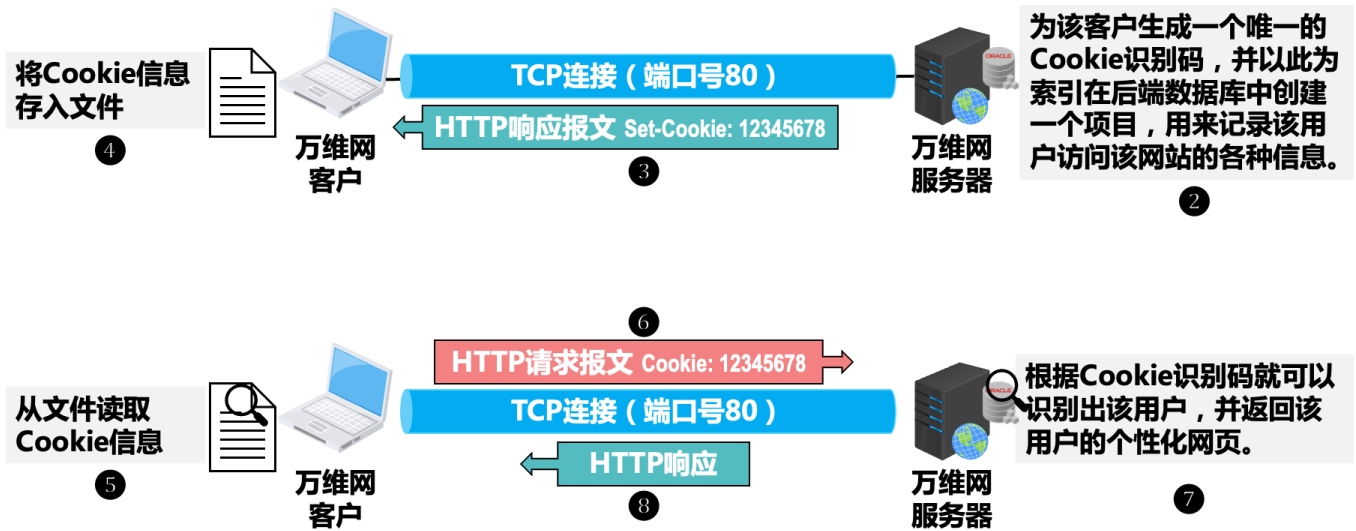
用户偏好、主题或者其他设置。

- 追踪

记录和分析用户行为。

Cookie 曾经用于一般的客户端存储。虽然这是合法的，因为它们是在客户端上存储数据的唯一方法，但如今建议使用现代存储 API。Cookie 随每个请求一起发送，因此它们可能会降低性能（尤其是对于移动数据连接而言）。

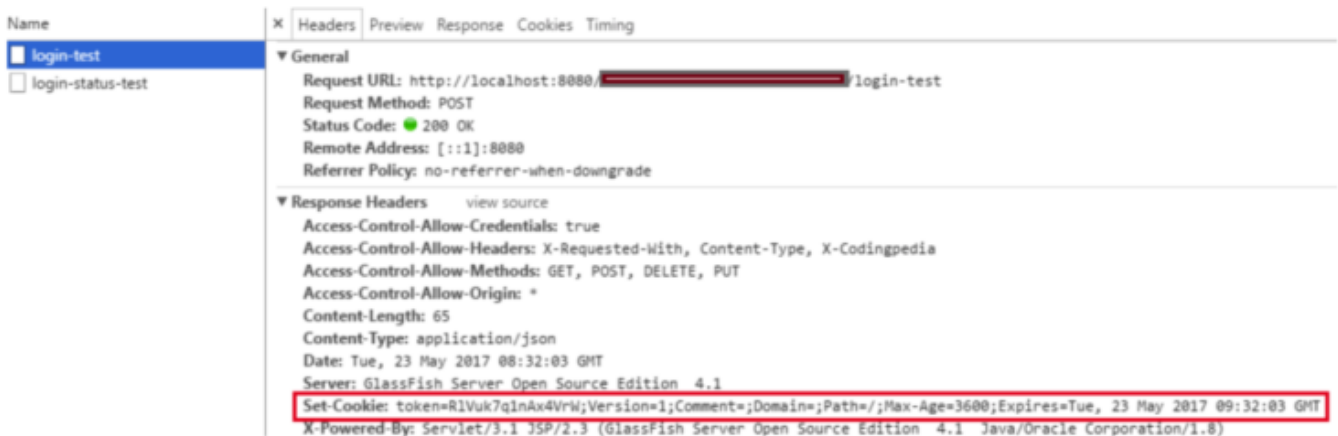
4.4.2. cookies的过程



当接收到客户端发出的 HTTP 请求时，服务器可以发送带有响应的 Set-Cookie 标头，Cookie 通常由浏览器存储，然后将 Cookie 与 HTTP 标头一同向服务器发出请求。

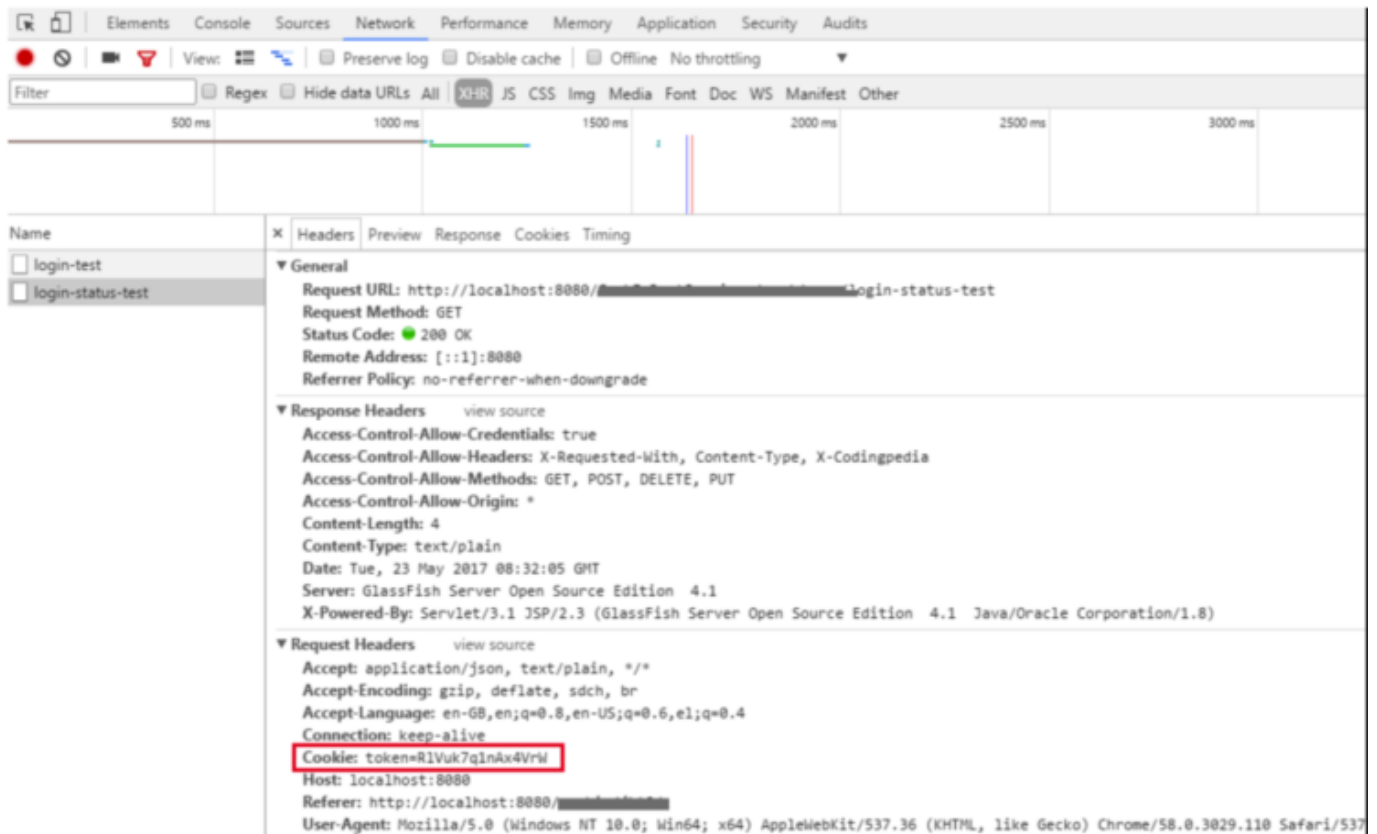
- Set-Cookie 和 Cookie 标头：

Set-Cookie HTTP 响应标头将 cookie 从服务器发送到用户代理。



此标头告诉客户端存储 Cookie。

现在，随着对服务器的每个新请求，浏览器将使用 Cookie 头将所有以前存储的 Cookie 发送回服务器。



有两种类型的 Cookies，一种是 Session Cookies，一种是 Persistent Cookies，如果 Cookie 不包含到期日期，则将其视为会话 Cookie。会话 Cookie 存储在内存中，永远不会写入磁盘，当浏览器关闭时，此后 Cookie 将永久丢失。如果 Cookie 包含有效期，则将其视为持久性 Cookie。在到期指定的日期，Cookie 将从磁盘中删除。

上面的示例创建的是会话 Cookie，会话 Cookie 有个特征，客户端关闭时 Cookie 会删除，因为它没有指定 Expires 或 Max-Age 指令。

但是，Web 浏览器可能会使用会话还原，这会使大多数会话 Cookie 保持永久状态，就像从未关闭过浏览器一样。

4.4.3. 永久性 Cookies

永久性 Cookie 不会在客户端关闭时过期，而是在特定日期（Expires）或特定时间长度（Max-Age）外过期。例如：

```
▼ 永久性 Cookies HTTP 复制代码
1 Set-Cookie: id=a3fWa; Expires=Wed, 21 Oct 2023 07:28:00 GMT;
```

- 对 Cookie 的争论

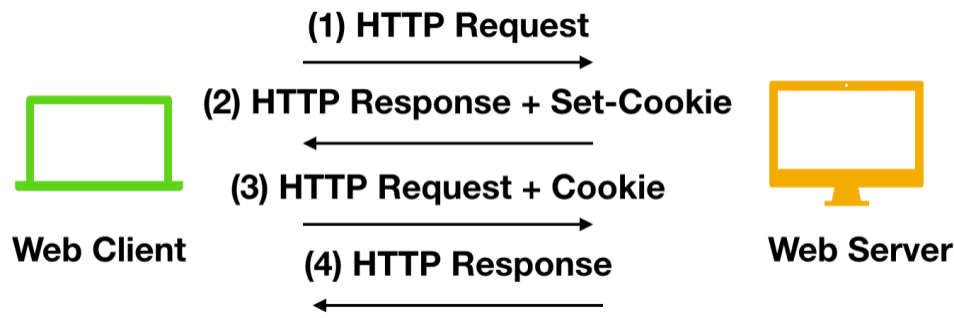
尽管 Cookie 能够简化用户的网络活动，但是 Cookie 的使用存在争议，因为不少人认为它对用户是一种侵权行为。因为结合 Cookie 和用户提供的账户信息，Web 站点可以知道更多关于用户的信息。

4.4.4. session是什么

客户端请求服务端，服务端会为这次请求开辟一块内存空间，这个对象便是 Session 对象，存储结构是一个 Map 映射，具体一点是 ConcurrentHashMap。Session 弥补了 HTTP 无状态特性，服务器可以利用 Session 存储客户端在同一个会话期间的一些操作记录。

- session如何判断是否是同一会话

服务器第一次接收到请求时，开辟了一块 Session 空间（创建了 Session 对象），同时生成一个 sessionId，并通过响应头的 Set-Cookie: JSESSIONID=XXXXXXX 命令，向客户端发送要求设置 Cookie 的响应；客户端收到响应后，在本机客户端设置了一个 JSESSIONID=XXXXXXX 的 Cookie 信息，该 Cookie 的过期时间为浏览器会话结束；



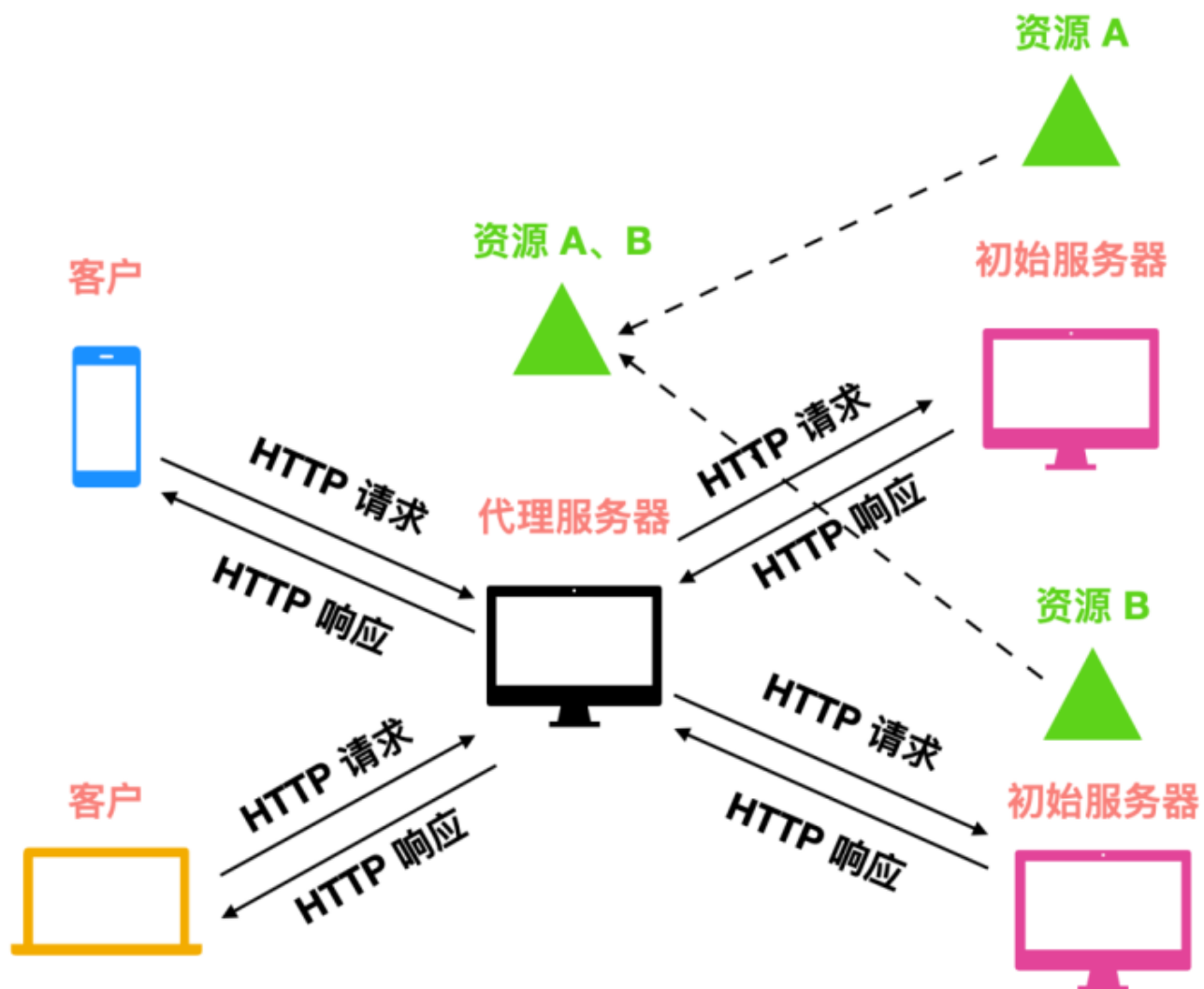
接下来客户端每次向同一个网站发送请求时，请求头都会带上该 Cookie信息(包含 sessionId)，然后服务器通过读取请求头中的 Cookie 信息，获取名称为 JSESSIONID 的值，得到此次请求的 sessionId。

session 机制有个缺点，比如 A 服务器存储了 session，就是做了负载均衡后，假如一段时间内 A 的访问量激增，会转发到 B 进行访问，但是 B 服务器并没有存储 A 的 session，会导致 session 的失效。

4.5. Web缓存

万维网缓存又称为Web缓存（Web Cache），可位于客户机，也可位于中间系统上，位于中间系统上的Web缓存又称为代理服务器（Proxy Server）。

Web 缓存 (Web cache) 也叫做代理服务器缓存，它是代表 HTTP 服务器来满足用户需求的网络实体。Web 缓存器有自己的磁盘存储空间，并会在存储空间内保存最近请求过的对象，如下图所示



Web 缓存可以在用户的浏览器中进行配置，一旦配置后，用户首先访问的就不是初始服务器了，需要先访问代理服务器判断请求的对象是否存在，如果代理服务器没有，再由代理服务器来请求初始服务器把对象返回给客户，同时在自己的磁盘空间保存对象。

这里需要注意，客户和初始服务器的架构是 客户-服务器模式，而代理服务器不仅能当服务器使用，也可以当作客户端使用。

代理服务器一般由 ISP (Internet Service Provider)，提供。ISP 也就是我们常说的运营商。

那么为什么需要代理服务器的存在呢？相信你看完上面的描述应该能大致猜到它的作用。

- 首先，代理服务器可以大大减少对客户请求的响应时间，能够更快给用户响应。
- 其次，代理服务器可以减少一个机构接入链路到网络的通信量，降低网络带宽，降低运营成本。
- 然后，代理服务器可以分担初始服务器的压力，改善应用程序的性能。

4.6. 练习

▼ 2015年题40

1. 某浏览器发出的HTTP请求报文如下：

```
GET /index.html HTTP/1.1
Host: www.test.edu.cn
Connection: Close
Cookie: 123456
```

下列叙述中，错误的是（ ）

- A. 该浏览器请求浏览index.html
 - B. index.html存放在www.test.edu.cn上
 - C. 该浏览器请求使用持续连接
 - D. 该浏览器曾经浏览过www.test.edu.cn
2. 假设HTTP1.1协议以持续的非流水线方式工作，一次请求-响应的时间为RTT，rfc.html页面引用了2个JPEG小图像，则浏览器从开始建立TCP连接到收到全部内容为止，需要多少个RTT？

